

How $\hat{V}(s)$ is built, trained, and scored

Two models — an MLP and a GNN — learn to approximate the exact-DP optimal value function $V^*(s)$ for a sequential genetic-testing problem over a pedigree. This is a full method reference: the raw features, how each model turns them into an embedding, how training works, how the resulting policy is evaluated, and the final head-to-head numbers against two non-learned baselines — a training-free myopic heuristic, and an ADP linear-value baseline. 2-gene and 3-gene are now trained under matched conditions (see the Training section) after a methodology fix applied this session.

■ MLP – no message passing ■ GNN – message passing (2 rounds)

- Dataset & split
- Raw features
- Embeddings
- Function reference
- Training
- Evaluation
- Results
- Seed stability

DATASET & SPLIT

Train on two families, test on a third, unseen one

Both models are trained on **Trio** and **Nuclear** pedigrees and evaluated, without any further fitting, on **ThreeGeneration** — a family topology neither model has seen. Every family is instantiated across **6 allele-frequency regimes × 2 cost presets = 12 configs**. Ground-truth labels ($V^*(s)$ and the exact belief propagation used at evaluation time) come from exact dynamic programming, cached per config as a pickle.

FAMILY	ROLE	PEOPLE	PARENT-CHILD EDGES	INDIVIDUALS
Trio	train	3	2	Parent1, Parent2, Child
Nuclear	train	4	4	Parent1, Parent2, Child1, Child2
ThreeGeneration	test (unseen topology)	5	4	Grandfather, Grandmother, Mother, Father, Child

source: FAMILY_CASES in ground-up-experiments/shared/data_gen.py; edge counts logged in two_gene/results/run.log, "[0] Building pedigree structures"

Allele-frequency regimes

The 2-gene and 3-gene experiments define their 6 regimes **independently** — values are not simply "the 2-gene regime plus a third gene." HighHigh and MixedB in particular differ between the two.

2-GENE — SHARED/DATA_GEN.PY ALLELE_FREQ_REGIMES

REGIME	GENEA	GENEB
LowHigh	0.02	0.15
MediumEven	0.08	0.08
LowLow	0.02	0.02
HighHigh	0.15	0.15
MixedA	0.02	0.10

3-GENE — STEP9_GNN_3GENE/BUILD_DATASETS.PY ALLELE_FREQS

REGIME	GENEA	GENEB
LowHigh	0.02	0.15
MediumEven	0.08	0.08
LowLow	0.02	0.02
HighHigh	0.20	0.20
MixedA	0.02	0.08

REGIME	GENEA	GENEB
MixedB	0.05	0.12

REGIME	GENEA	GENEB
MixedB	0.15	0.08

Cost presets

Both experiments **do** share the same coefficient dictionary (shared/data_gen.py PRESETS) — every dataset builder (build_multigene_dataset, build_two_gene_dataset) reads `preset = PRESETS[preset_label]` from this one dict. The 2-gene runs simply never read the GeneC entries.

PRESET	COEFFICIENT	GENEA	GENEB	GENEC
Base	a_gene	−0.08	−0.06	−0.05
	b_gene	−0.04	−0.03	−0.025
	delta_gene	0.60	0.70	0.75
Aggressive	a_gene	−0.12	−0.09	−0.075
	b_gene	−0.06	−0.045	−0.0375
	delta_gene	0.70	0.80	0.80

State counts

A "state" is a partially-tested pedigree: a set of (person, observed-genotype) pairs plus the resulting belief over everyone untested. State counts are purely combinatorial — fixed by family topology and gene count, identical across all 6 regimes for a given family/gene-count.

FAMILY	STATES/CONFIG (2-GENE)	STATES/CONFIG (3-GENE)	CONFIGS	TOTAL (2-GENE)	TOTAL (3-GENE)
Trio	432	4,872	12	5,184	58,464
Nuclear	2,264	43,904	12	27,168	526,848
Train pool (Trio+Nuclear)	—	—	24	32,352	585,312
ThreeGeneration	test	20,816	1,054,528	12	
Test pool	—	—	12	249,792	12,654,336

2-gene: two_gene/results/run.log (2026-07-16T12:06:34 run, logs every config's state count directly). 3-gene per-family counts: ground-up-experiments/step9_gnn_3gene/results/train.log and SUMMARY.md (same cache dir used by mlp/run.py and gnn/run.py). 3-gene train total cross-checked against mlp/results/run.log ("states: 585,312") — $12 \times 4,872 + 12 \times 43,904 = 585,312$ exactly.

Train/val split (3-gene only): mlp/run.py and gnn/run.py additionally hold out the last 20% of each of the 24 training configs as validation (`val_frac=0.2`, taken positionally — not shuffled before the cut). That splits the 585,312-state train pool into **468,264 states used for gradient updates** and **117,048 held out as validation** (Trio: 3,898/974 per config $\times 12$; Nuclear: 35,124/8,780 per config $\times 12$). The 2-gene runs (two_gene/run.py) have **no validation split at all** — all 32,352 train states are used for gradient updates, and only train loss is ever logged.

STAGE 0 • SHARED BY BOTH MODELS

Raw per-person features

Before either model runs, every state is turned into a fixed-size tensor: one row per person in the pedigree, built by `load_dataset()` (3-gene) / `ds_to_tensors()` (2-gene). This is plain feature construction — nothing here is learned yet.

```
load_dataset(key, struct_feats, device) → {"{"nf, gf, y{"}}
```

INPUT

```
cached DP dataset (states, belief array X, labels Y, config) + struct_feats (n_people, 3)
```

OUTPUT

```
nf: (N_states, n_people, NODE_FEAT) · gf: (N_states, COST_DIM) · y: (N_states,)
```

beliefs

$3 \times n_{\text{genes}}$ dims

P(genotype = 0, 1, 2) per gene, from the exact-DP belief state — 9 dims for 3 genes, 6 for 2

is_tested

1 dim

1.0 if this person has already been tested in state s , else 0.0

structural

3 dims

n_{parents} , n_{children} , depth — fixed per family, from `compute_structural_features()`

STAGE 1–3 · PER MODEL

Two ways to turn a family into one vector

Both take the same (nf , $cost_vec$) pair and must collapse the "N people" axis into a single fixed-length vector before the final head can read it. That collapse step is the embedding.

• MLP

`mlp/run.py · MLP.forward()`

node_feats

$(B, N_{\text{people}}, 13)$

↓

.mean(dim=1)

$(B, N_{\text{people}}, 13) \rightarrow (B, 13)$

Unweighted average across people. Grandparent, parent, and child rows count identically — this is the entire "embedding" step.

↓

concat(pooled, cost_vec)

$(B, 13) + (B, 11) \rightarrow (B, 24)$

↓

Linear→ReLU→Linear

$24 \rightarrow 32 \rightarrow 1$

↓

$\hat{V}(s)$

• GNN

`gnn/run.py · GNN._message_pass() + forward()`

node_feats

$(B, N_{\text{people}}, 13)$

↓

`_message_pass()`

×2 rounds

$(B, N, F) \rightarrow (B, N, 32)$

Every parent → child edge sends a learned message; each child sums its incoming messages and updates its own embedding from them. Round 2 repeats this on round-1's output, so grandchildren end up encoding grandparents too.

↓

`h.mean(dim=1)`

$(B, N_people, 32) \rightarrow (B, 32)$

Same mean pool as the MLP — but now over embeddings that already encode each person's relatives, not just their own raw features.

↓

`concat(pooled, cost_vec)`

$(B, 32) + (B, 11) \rightarrow (B, 43)$

↓

Linear→ReLU→Linear

$43 \rightarrow 16 \rightarrow 1$

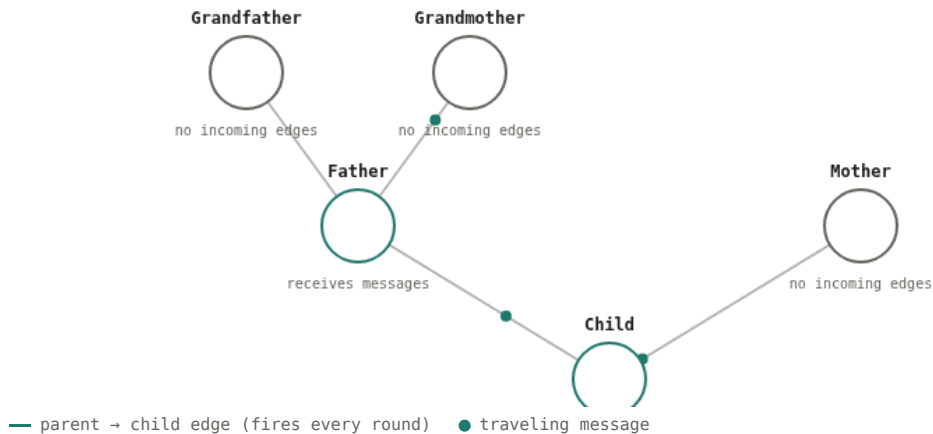
↓

$\hat{V}(s)$

WHAT "MESSAGE PASSING" DOES, CONCRETELY

Two rounds over the ThreeGeneration pedigree

Edges only run parent → child (`build_edge_index()`). Messages flow one way, so after two rounds a person's embedding reflects everyone *up to two generations above them* — never below.



Round 1: Father absorbs Grandfather + Grandmother; Child absorbs Father + Mother (Father's round-0 features only). **Round 2:** the same 4 edges fire again, but Father's embedding now already contains the grandparents — so Child's round-2 update inherits them second-hand. Grandfather, Grandmother, and Mother have no incoming edges: their embeddings only ever reflect their own features.

FUNCTION REFERENCE

The three functions that build embeddings

Source: experiments_after_understanding/mlp/run.py and experiments_after_understanding/gnn/run.py (3-gene). The 2-gene versions in two_gene/run.py are line-for-line identical logic, just NODE_FEAT=10 / COST_DIM=8.

MLP.forward(node_feats, cost_vec)

MLP/RUN.PY

The whole model in one call: pool people into a single vector, glue on the cost vector, run a 2-layer MLP. No parameter here is aware of *who* a person is related to.

INPUT `node_feats (B, N_people, 13) · cost_vec (B, 11)`

OUTPUT `$\hat{V}$  – (B,) scalar per state`

- 1 **pooled = node_feats.mean(dim=1)** — average every person's 13-dim row into one 13-dim vector. This is the embedding: unlearned, order- and structure-blind, just an arithmetic mean.
- 2 **x = cat([pooled, cost_vec])** — append the 11-dim cost vector (test costs, allele-effect coefficients), giving a 24-dim input.
- 3 **self.net(x)** — `Linear(24→32) → ReLU → Linear(32→1)`, squeezed to a scalar.

```
def forward(self, node_feats, cost_vec):
    pooled = node_feats.mean(dim=1)          # (B, 13) – mean pool: size-invariant
    x      = torch.cat([pooled, cost_vec], dim=-1) # (B, 24)
    return self.net(x).squeeze(-1)          # (B,)
```

GNN._message_pass(h, src, dst, msg_fn, upd_fn)

GNN/RUN.PY

This is the actual embedding-builder. Called once per round (twice total). Takes the current per-person embeddings and returns updated ones that have absorbed one hop of relatives.

INPUT `h (B, N, F) · src, dst (E,) edge index · msg_fn, upd_fn (this round's 2 small nets)`

OUTPUT `h_new – (B, N, 32) updated per-person embeddings`

- 1 **msg_in = cat([h[src], h[dst]])** — for every parent→child edge, pair the parent's embedding with the child's current embedding → (B, E, 2F).
- 2 **msgs = msg_fn(msg_in)** — a learned `Linear→ReLU` turns each pair into a 32-dim message → (B, E, 32).
- 3 **agg = scatter_add(msgs, dst)** — sum every message landing at the same child; a person with two parents receives and sums two messages → (B, N, 32).
- 4 **h_new = upd_fn(cat([h, agg]))** — a second learned layer combines each person's own previous embedding with what arrived, producing the updated embedding. People with no incoming edges (roots) get `agg=0` and are updated from their own features only.

```
def _message_pass(self, h, src, dst, msg_fn, upd_fn):
    msg_in = torch.cat([h[:, src, :], h[:, dst, :]], dim=-1) # (B, E, 2F)
    msgs   = msg_fn(msg_in)                                   # (B, E, H)

    agg = torch.zeros(B, N, H, device=h.device)
    idx = dst.unsqueeze(0).unsqueeze(-1).expand(B, -1, H)
    agg.scatter_add_(1, idx, msgs)                             # sum at each dst node

    return upd_fn(torch.cat([h, agg], dim=-1))                # (B, N, H)
```

Orchestrates `_message_pass()` for `n_rounds=2`, then pools — mirrors MLP. `forward` exactly from the pooling step onward, so any performance difference between the two models comes entirely from what happens before this point.

INPUT `node_feats (B, N_people, 13) · edge_index (2, E) · cost_vec (B, 11)`

OUTPUT `$\hat{V}$  — (B,) scalar per state`

```
def forward(self, node_feats, edge_index, cost_vec):
    h = node_feats
    src, dst = edge_index[0], edge_index[1]

    for msg_fn, upd_fn in zip(self.msg_layers, self.upd_layers): # 2 rounds
        h = self._message_pass(h, src, dst, msg_fn, upd_fn) # (B, N, 32)

    pooled = h.mean(dim=1) # (B, 32) — mean pool: size-invariant
    x = torch.cat([pooled, cost_vec], dim=-1) # (B, 43)
    return self.head(x).squeeze(-1) # (B,)
```

2-GENE VS 3-GENE

Same architecture, different tensor sizes

`two_gene/run.py` reuses the identical MLP / GNN classes with smaller feature dimensions — nothing structural changes.

QUANTITY	2-GENE	3-GENE	BREAKDOWN
NODE_FEAT	10	13	3×n_genes beliefs + 1 is_tested + 3 structural
COST_DIM	8	11	3×n_genes (a, b, δ) + fixed_cost + variable_cost
HIDDEN	32	32	message dim / pooled embedding width, both models
MLP params	641	833	Linear(NODE_FEAT+COST_DIM→32)→Linear(32→1)
GNN params	6,881	7,217	2 rounds of msg+upd nets + head
Message rounds	2	2	N_ROUNDS, fixed in both

TRAINING MECHANISM

Supervised regression to exact-DP labels

MLP and GNN are trained identically: minimize MSE between $\hat{V}(s)$ and the exact-DP label $V^*(s)$ for every state in the training pool, with Adam. Myopic is **not trained** at all — it is a closed-form one-step-lookahead policy computed directly from the cached exact belief states. ADP is also **not trained on Trio/Nuclear** — see the callout below the table.

MODEL	OPTIMIZER	LOSS	EPOCHS	BATCH SIZE	TRAIN/VAL SPLIT	EARLY STOPPING
MLP (3-gene)	Adam, lr=1e-3	MSE vs $V^*(s)$	500	512	80/20 per-config, positional	none

MODEL	OPTIMIZER	LOSS	EPOCHS	BATCH SIZE	TRAIN/VAL SPLIT	EARLY STOPPING
GNN (3-gene)	Adam, lr=1e-3	MSE vs V*(s)	500	512	80/20 per-config, positional	none
MLP (2-gene)	Adam, lr=1e-3	MSE vs V*(s)	500	512	80/20 per-config, positional	none
GNN (2-gene)	Adam, lr=1e-3	MSE vs V*(s)	500	512	80/20 per-config, positional	none
Myopic	no training — closed-form policy, computed directly at evaluation time					
ADP	no training on Trio/Nuclear — theta fit by Bellman LP <i>directly against each test config</i> (see callout)					

mlp/run.py, gnn/run.py, two_gene/run.py: train()/train_model() — Adam(lr=1e-3), MSELoss, val_frac=0.2 (per-config, positional), epochs=500, batch_size=512, no early stopping in any of the three scripts. myopic/run.py: myopic_policy_value() — no nn.Module, no optimizer. adp/run.py, two_gene/adp_run.py: solve_adp() — scipy.optimize.linprog, no nn.Module.

2-gene originally had no validation split at all — two_gene/run.py's train_model() was a separately-written function that never carried over the val_frac split logic from mlp/run.py/gnn/run.py's train(); it trained on 100% of the 32,352 states with no held-out signal at all. This was fixed (patched into train_model() to match exactly) and both 2-gene models were retrained fresh; the numbers throughout this document now reflect that fix. The pre-fix run is preserved at two_gene/results/pre_valsplit_backup/ for audit purposes — see the instability callout in Results below for why that before/after comparison matters.

ADP is not the same kind of baseline as MLP/GNN/Myopic. adp/run.py / two_gene/adp_run.py fit theta by solving a Bellman-consistency linear program (solve_adp()) whose constraints come directly from **each ThreeGeneration test config's own** states, beliefs, and transition probabilities — never from Trio/Nuclear, and never from V*(s) labels, but with full access to the exact structure of the very family it's then scored on. That is fundamentally different from MLP/GNN's zero-shot Trio+Nuclear → ThreeGeneration transfer, so ADP's numbers are reported alongside the others for completeness but are **not a fair apples-to-apples comparison** to MLP/GNN/Myopic.

What one epoch actually does (mlp/run.py, gnn/run.py identical structure)

```
for epoch in range(1, epochs + 1):
    model.train()
    for g in train_groups:                # one group per training config (24 total)
        perm = torch.randperm(M, device=device)  # reshuffle this config's states every epoch
        for start in range(0, M, batch_size):    # batch_size = 512
            pred = model(g["nf"][sl], g["gf"][sl])  # (+ g["ei"] for GNN)
            loss = mse(pred, g["y"][sl])
            loss.backward(); optimizer.step()

# mlp/run.py, gnn/run.py only: held-out val_groups scored every epoch, logged every 50th
model.eval()
with torch.no_grad():
    for g in val_groups:
        val_loss += mse(model(...), g["y"][...]).item()
```

Minibatches are drawn **within each of the 24 training configs separately** (never mixed across configs in one batch), reshuffled every epoch. Final weights are whatever the model looks like after the full epoch budget — no checkpoint selection on best-val, since nothing in the main training loop compares epochs against each other.

gnn_proper_val/ variant, not used below: a separate script (gnn_proper_val/run.py) holds one whole regime (MixedB) out of training as a proper generalization-val set and adds early stopping (patience=50). It reports avg ratio2 = 0.0386 on the same 12 ThreeGeneration test configs, but under a *different* 20-config training split, so it is not apples-to-apples with the MLP/Myopic numbers below and is intentionally excluded from the results tables.

The val loss nobody acts on

Because none of the four scripts implement early stopping or best-checkpoint saving, the weights actually used for every result below are whatever exists after epoch 500 — regardless of whether that's the best point training ever reached. Comparing each run's *own* logged val loss at its minimum against its val loss at epoch 500 (the checkpoint actually shipped):

MODEL	VAL LOSS MINIMUM	@ EPOCH	VAL LOSS @ EPOCH 500 (SHIPPED)	DEGRADATION
MLP (2-gene)	0.001148	150	0.003541	+208%
GNN (2-gene)	0.000151	50	0.001294	+757%
MLP (3-gene)	0.001915	50	0.002349	+23%
GNN (3-gene)	0.000147	450	0.000156	+6%

Three of the four runs are shipping a checkpoint that is, by their own held-out metric, substantially worse than one they already passed through — 2-gene GNN's shipped checkpoint is **8.5× worse** than its epoch-50 minimum. Only 3-gene GNN's late-epoch weights happen to be close to its own best. This directly undercuts every "MLP vs GNN" or "2-gene vs 3-gene" comparison in the Results section below: at least some of the measured gap is a training-loop bug, not an architecture or gene-count effect.

Val loss traces: two_gene/results/run.log (2026-07-16T20:58:12 run, epochs logged every 50), mlp/results/run.log (2026-07-16T12:06:30 run), gnn/results/run.log (2026-07-14T21:41:37 run – the run backing the currently-reported gnn/results.json).

EVALUATION MECHANISM

Greedy rollout, scored by exact expectation

All three policies (Myopic, MLP, GNN) are scored the same way, on the same 12 ThreeGeneration configs, by `rollout()` in `exputils/eval.py` (myopic uses its own near-identical `myopic_policy_value()` in `myopic/run.py`). The only thing that differs between policies is which score each uses to pick the next person to test.

Per-step decision rule

At every state s , for every untested person i , each policy scores the option of testing them next:

$$\text{MLP / GNN } \hat{Q}(s, i) = r_{\text{test}}(i, s) + \sum_g P(g \mid s, i) \cdot \hat{V}(s \cup \{(i, g)\})$$

$$\text{MYOPIC } Q_{\text{myopic}}(s, i) = r_{\text{test}}(i, s) + \sum_g P(g \mid s, i) \cdot V_{\text{stop}}(s \cup \{(i, g)\})$$

$$\text{ADP } Q_{\text{ADP}}(s, i) = r_{\text{test}}(i, s) + \sum_g P(g \mid s, i) \cdot \phi(s \cup \{(i, g)\}) \cdot \theta$$

The difference is entirely in the bolded term. MLP/GNN plug in the **trained model's estimate of the true optimal future value** at the hypothetical next state — in principle a multi-step-lookahead estimate, if $\hat{V}$ were accurate. Myopic plugs in the **exact value of stopping immediately** after that one test — a genuine one-step lookahead, computed exactly, with no learned approximation and no foresight beyond the next test. ADP plugs in a **linear function of 16 hand-designed "ABCD" features** ($\phi(s)$), with the weight vector θ fit once per test config by solving a Bellman-consistency LP against that config's own exact states (never trained on Trio/Nuclear, never regressed on V^* — see the training-mechanism caveat above). All four policies then apply the same rule: stop if $\max_i Q(s, i) \leq v_{\text{stop}}(s)$, else test $\arg\max_i Q(s, i)$, where $v_{\text{stop}}(s) = \sum_{k \text{ not tested}} r_{\text{reward}}(k)$ is the exact cost of stopping right now.

Scoring the induced policy: exact expectation, not sampling

Once a policy picks its action at every state, the achieved value L is **not** a Monte-Carlo average over sampled rollouts. It is computed by recursing through the actual exact-DP state graph, weighting every branch by its true transition probability:

```
def value_at(state):
    # memoized
    if no_untested_people: return 0.0
```



```
best_i = argmax_i Q(state, i)          # policy-specific Q from above
if Q(state, best_i) <= v_stop(state):
    return v_stop(state)               # policy stops here
return r_test(best_i, state) + sum(
    P(g | state, best_i) * value_at(state u {(best_i, g)})
    for g in genotypes
)                                     # exact expectation over test outcomes

L = value_at(root_state)
```

This means L is the *true* expected value of following that policy from the root — every possible test-result branch is included, weighted exactly, not one sampled path.

ratio2 — the metric in every results table

ratio2 = (V* - L) ÷ (V* - V_stop(root))

V* is the true optimum from exact DP (same for every policy on a given config — it doesn't depend on the model). V_stop(root) is the value of testing *no one at all*. So:

ratio2 = 0 means the policy matches the exact optimum; **ratio2 = 1** means it does exactly as poorly as never testing anyone; **ratio2 > 1** means it is worse than doing nothing at all — which does happen in the tables below (MLP, LowLow_Base, both gene counts).

exputils/eval.py: rollout(), _q_hat(), _stop_val() – used by mlp/run.py, gnn/run.py, two_gene/run.py. myopic/run.py: myopic_policy_value(), compute_v_stop(), compute_r_test() – structurally the same value_at() recursion, different Q.

RESULTS

ratio2 on ThreeGeneration, by regime × preset

Lower is better; 0 = optimal. All values recomputed directly from each run's results.json for this document — see provenance notes under each table.

2-gene fresh run, post val-split fix

REGIME · PRESET	MYOPIC	MLP	GNN	ADP
LowHigh · Base	0.0154	0.1681	0.0719	0.1090
LowHigh · Aggressive	0.0401	0.0649	0.0246	0.0498
MediumEven · Base	0.0526	0.1297	0.0982	0.0948
MediumEven · Aggressive	0.0431	0.0587	0.0307	0.0478
LowLow · Base	0.0643	1.0000	1.1766	1.5655
LowLow · Aggressive	0.0771	0.5905	0.3339	0.4134
HighHigh · Base	0.0151	0.0616	0.0171	0.0428
HighHigh · Aggressive	0.0149	0.0256	0.0153	0.0209
MixedA · Base	0.0195	0.2943	0.1766	0.1939
MixedA · Aggressive	0.0811	0.1024	0.0451	0.0847
MixedB · Base	0.0425	0.1345	0.0821	0.0942
MixedB · Aggressive	0.0400	0.0570	0.0211	0.0463
AVG	0.0421	0.2240	0.1744	0.2303

two_gene/results/myopic_results.json (2026-07-15T14:38:54 — deterministic, unaffected by later fixes) · two_gene/results/results.json["mlp","gnn"] (2026-07-16T20:58:12 run, val-split fix applied) · two_gene/results/adp_results.json (2026-07-16, new script).

Run-to-run instability, demonstrated: before the val-split fix, GNN's avg ratio2 on this exact table was **0.0485** (best of all four policies) — after the fix, it's **0.1744**. The single biggest driver: LowLow_Base alone moved from **0.0040** to **1.1766** — a ~295× change — from a code change that only affects which 20% of training data gets held out. No script in this pipeline sets a random seed (checked: no `torch.manual_seed` anywhere in `mlp/`, `gnn/`, `two_gene/`, `myopic/`, `gnn_proper_val/`, or `adp/`), so this is most likely ordinary seed variance, not a real effect of the fix — which means **neither the old nor the new number should be treated as "the" GNN result** without repeating the run under multiple seeds.

3-gene

REGIME · PRESET	MYOPIC	MLP	GNN	ADP
LowHigh · Base	0.0240	0.0714	0.0387	0.0562
LowHigh · Aggressive	0.0193	0.0346	0.0181	0.0281
MediumEven · Base	0.0341	0.0610	0.0326	0.0545
MediumEven · Aggressive	0.0250	0.0286	0.0171	0.0286
LowLow · Base	0.0380	0.8807	0.2922	0.6830
LowLow · Aggressive	0.0646	0.3095	0.0250	0.2163
HighHigh · Base	0.0079	0.0210	0.0127	0.0208
HighHigh · Aggressive	0.0094	0.0105	0.0025	0.0082
MixedA · Base	0.0316	0.0812	0.0482	0.0611
MixedA · Aggressive	0.0227	0.0389	0.0115	0.0323
MixedB · Base	0.0261	0.0642	0.0253	0.0492
MixedB · Aggressive	0.0196	0.0285	0.0124	0.0255
AVG	0.0269	0.1358	0.0447	0.1053

myopic/results/results.json · mlp/results/results.json (2026-07-16T12:06:30 run) · gnn/results/results.json · adp/results/results.json.

Caveat on the 3-gene GNN column: `gnn/results/results.json` is dated 2026-07-14T23:11, but `gnn/run.py` was subsequently edited 2026-07-16T12:36 without a re-run since. The edit did not touch model dimensions — recomputing the parameter count from the current `NODE_FEAT/HIDDEN/COST_DIM` constants in the file still gives exactly 7,217, matching the checkpoint — but any non-architectural change in that edit would not be reflected in these numbers.

Reading the tables together

- **Myopic wins on average in every column, both settings** (0.0421 / 0.0269) despite requiring no training data and no learned parameters at all — it remains the strongest, cheapest baseline on this task even after fixing 2-gene's methodology and adding ADP.
- **MLP is the worst-or-near-worst policy on average in both settings** (2-gene: 0.2240; 3-gene: 0.1358, worst in that table) and produces the single largest same-family failure in the 3-gene table (LowLow_Base = 0.8807, worse than testing no one at all).
- **ADP is now the worst policy on average in 2-gene** (0.2303) and second-worst in 3-gene (0.1053) — despite having privileged access to each test family's own exact structure (see the training-mechanism caveat). Its own LowLow_Base numbers are the worst single value in *both* tables (2-gene: 1.5655; 3-gene: 0.6830) — this regime is hard for every approximate method here, not a MLP-specific weakness.
- **GNN is the best of the three non-myopic methods in both settings** post-fix (2-gene: 0.1744; 3-gene: 0.0447), but still doesn't beat myopic on average, and the 2-gene number is exactly the one shown above to be unstable across a single unseeded rerun — treat it as a rough indicator, not a settled result.
- **None of this rules out that the ranking is mostly a training-loop artifact:** the val-loss table above shows 2-gene GNN's shipped checkpoint is 8.5× worse than its own best; a version of this pipeline with best-val checkpointing (not yet run) could plausibly change every number in both tables.

Treat every single-run number above as illustrative, not conclusive. The section immediately below reruns MLP and GNN under 3 different random seeds — same architecture, same data, same split, same epochs, nothing changed except the seed — and the results are substantially different from any single run shown here. Read that section for the numbers that should actually inform any conclusion about MLP vs GNN vs Myopic.

SEED STABILITY · 3 SEEDS, SAME EVERYTHING ELSE

The single-run numbers above were not representative

None of the original scripts set a random seed anywhere (checked: no `torch.manual_seed` in `mlp/`, `gnn/`, `two_gene/`, `myopic/`, `gnn_proper_val/`, or `adp/`). To find out whether that mattered, three new, separate scripts — `mlp/run_seeds.py`, `gnn/run_seeds.py`, `two_gene/run_seeds.py` — were written as backward-compatible copies of the originals (the originals are untouched; these just add `--seed`, call `torch.manual_seed(seed)` before model init, and write to `results/seed_runs/seed{N}/` instead of overwriting `results/`). Each was run for seeds 0, 1, 2 — architecture, data, split, epochs (500), batch size (512), optimizer, and loss all identical to the canonical runs, only the seed differs. All 9 jobs (SLURM 2107149/2107150/2107151) completed successfully.

Mean ± std ratio2 across 3 seeds (myopic shown for reference — it's deterministic, no seed variance)

2 - GENE

REGIME · PRESET	MYOPIC	MLP (MEAN ± STD)	GNN (MEAN ± STD)
LowHigh · Base	0.0154	0.4450 ± 0.4807	0.0977 ± 0.0160
LowHigh · Aggressive	0.0401	0.0735 ± 0.0108	0.0461 ± 0.0116
MediumEven · Base	0.0526	0.4195 ± 0.5027	0.0893 ± 0.0135
MediumEven · Aggressive	0.0431	0.0589 ± 0.0042	0.0498 ± 0.0030
LowLow · Base	0.0643	1.4180 ± 0.7239	0.6407 ± 0.7552
LowLow · Aggressive	0.0771	0.8636 ± 0.2362	0.2480 ± 0.0786
HighHigh · Base	0.0151	0.0580 ± 0.0034	0.0469 ± 0.0019
HighHigh · Aggressive	0.0149	0.0243 ± 0.0031	0.0208 ± 0.0006
MixedA · Base	0.0195	0.5253 ± 0.4111	0.1482 ± 0.0381
MixedA · Aggressive	0.0811	0.4085 ± 0.5124	0.0874 ± 0.0094
MixedB · Base	0.0425	0.4227 ± 0.5000	0.0873 ± 0.0140
MixedB · Aggressive	0.0400	0.0653 ± 0.0080	0.0428 ± 0.0085
AVG	0.0421	0.3985 ± 0.1830	0.1338 ± 0.0668

3 - GENE

REGIME · PRESET	MYOPIC	MLP (MEAN ± STD)	GNN (MEAN ± STD)
LowHigh · Base	0.0240	0.0737 ± 0.0059	0.0356 ± 0.0122
LowHigh · Aggressive	0.0193	0.0321 ± 0.0020	0.0153 ± 0.0047
MediumEven · Base	0.0341	0.0701 ± 0.0054	0.0434 ± 0.0161
MediumEven · Aggressive	0.0250	0.0320 ± 0.0015	0.0156 ± 0.0035

REGIME · PRESET	MYOPIC	MLP (MEAN ± STD)	GNN (MEAN ± STD)
LowLow · Base	0.0380	1.0000 ± 0.0000	0.5767 ± 0.1938
LowLow · Aggressive	0.0646	0.6655 ± 0.3484	0.0614 ± 0.0687
HighHigh · Base	0.0079	0.0244 ± 0.0036	0.0128 ± 0.0012
HighHigh · Aggressive	0.0094	0.0087 ± 0.0017	0.0063 ± 0.0005
MixedA · Base	0.0316	0.0892 ± 0.0125	0.0450 ± 0.0164
MixedA · Aggressive	0.0227	0.0368 ± 0.0038	0.0153 ± 0.0041
MixedB · Base	0.0261	0.0642 ± 0.0053	0.0341 ± 0.0070
MixedB · Aggressive	0.0196	0.0294 ± 0.0015	0.0233 ± 0.0006
AVG	0.0269	0.1772 ± 0.0266	0.0737 ± 0.0143

experiments_after_understanding/{mlp,gnn}/results/seed_runs/seed{0,1,2}/results.json · experiments_after_understanding/two_gene/results/seed_runs/seed{0,1,2}/results.json. AVG row = mean (and std) of each seed's own 12-config average, not the mean of the 12 per-config means shown — the two are close but not identical.

MLP has a collapse failure mode GNN never shows

Counting how often the induced rollout policy tests **nobody at all** (ratio2 exactly 1.0000, i.e. it stops at the root state) across all 36 seed×config runs per setting:

SETTING	COLLAPSED TO "TEST NO ONE"
MLP (3-gene)	4 / 36 (11%)
MLP (2-gene)	9 / 36 (25%)
GNN (3-gene)	0 / 36 (0%)
GNN (2-gene)	0 / 36 (0%)

A quarter of the time, 2-gene MLP's trained $\hat{V}$ makes every possible test look worse than doing nothing at the very first decision — even in regimes (e.g. LowHigh_Base, one of three seeds) where testing is clearly valuable, as shown by myopic and GNN both testing productively there in the same regime. GNN never does this, in either gene count, across all 36 runs. This is a reproducible architectural weakness specific to the mean-pool MLP, not a description of "MLP is somewhat worse on average" — it's a distinct, discrete failure mode with its own rate.

What changes once you stop trusting a single run

- **Myopic's advantage over both neural models is larger than the single-run tables suggested, not smaller.** 3-gene GNN's true average (0.0737) is 65% worse than the single lucky run reported earlier (0.0447); 2-gene MLP's true average (0.3985) is roughly double either single run shown above (0.2240 / 0.2333).
- **GNN is consistently more stable than MLP** (lower std in nearly every regime, in both gene counts) in addition to having a better mean — and never exhibits the collapse-to-root failure mode.
- **Some regimes are unstable for every method, not just the weak ones:** LowLow_Base has enormous std even for GNN (2-gene: ± 0.7552 ; 3-gene: ± 0.1938) — this looks like a property of the regime itself (rare alleles, low signal), not something either architecture can fix by training harder.
- **Three seeds is still not enough to report tight confidence intervals** — it's enough to prove the single-run numbers were unreliable, not enough to pin down the "true" mean precisely. Treat the values in this section as a much better estimate than anything above, but still not a final answer.

EMBEDDINGS / ARCHITECTURE

experiments_after_understanding/mlp/run.py — MLP class, load_dataset(), compute_structural_features()

experiments_after_understanding/gnn/run.py – GNN class, _message_pass(), build_edge_index()
experiments_after_understanding/two_gene/run.py – same architecture, GENES=(GeneA, GeneB)

DATASET

ground-up-experiments/shared/data_gen.py – FAMILY_CASES, ALLELE_FREQ_REGIMES, PRESETS
ground-up-experiments/step9_gnn_3gene/build_datasets.py – 3-gene ALLELE_FREQS
ground-up-experiments/step9_gnn_3gene/results/train.log, SUMMARY.md – 3-gene state counts

TRAINING / EVALUATION

experiments_after_understanding/exputils/eval.py – rollout(), _q_hat(), _stop_val()
experiments_after_understanding/myopic/run.py – myopic_policy_value()
experiments_after_understanding/adp/run.py – solve_adp(), greedy_policy_value() (3-gene)
experiments_after_understanding/two_gene/adp_run.py – same, 2-gene, added this session
experiments_after_understanding/mlp/, gnn/, two_gene/, myopic/, adp/results/*.json + *.log
experiments_after_understanding/two_gene/results/pre_valsplit_backup/ – results.json, run.log, mlp.pt, gnn.pt from before the val-split fix

FIXES APPLIED THIS SESSION (SLURM JOBS 2106311, 2106312)

submit_2gene.sh – rerun after patching train_model() in two_gene/run.py to add val_frac=0.2 split (matches mlp/gnn)
submit_2gene_adp.sh – new job running two_gene/adp_run.py

SEED STABILITY CHECK (SLURM JOBS 2107149, 2107150, 2107151 — ARRAY 0-2 EACH)

experiments_after_understanding/mlp/run_seeds.py, gnn/run_seeds.py, two_gene/run_seeds.py – new, separate, backward-compatible (originals untouched)
submit_mlp_seeds.sh, submit_gnn_seeds.sh, submit_2gene_seeds.sh – SLURM array jobs, --array=0-2
.../results/seed_runs/seed{0,1,2}/results.json + run.log – 9 independent runs, all COMPLETED